System Programming

Advanced Procedures

Dongwann Kang

Associate Professor
Department of Computer Science and Engineering

Stack Frames

- 1. Stack Parameters
  - Stack frame
    - The area of the stack set aside for the followings:
      - Passed arguments
      - Subroutine return address
      - Local variables
      - Saved registers
  - The stack frame is created by the following steps:
    1. Passed arguments, if any, are pushed on the stack
    2. The subroutine is called, causing the subroutine return address to be pushed on the stack
    3. As the subroutine begins to execute, EBP is pushed on the stack
    4. EBP is set equal to ESP
       - From this point on, EBP acts as a base reference for all of the subroutine parameters
    5. If there are local variables, ESP is decremented to reserve space for the variables on the stack
    6. If any registers need to be saved, they are pushed on the stack
  - The structure of a stack frame is affected by the memory model and argument passing convention
  - If you want to call functions in the 32-bit Windows API, you must pass arguments on the stack

Stack Frames

- 2. Disadvantages of Register Parameters
  - Microsoft's *fastcall* parameter passing convention
    - Simply placing parameters in registers before calling a subroutine
      - This leads to runtime efficiency
    - Unfortunately, these registers are used to hold data values
    - Therefore, any registers used as parameters must first be pushed on the stack before procedure calls, and later restored to their original values after the procedure returns

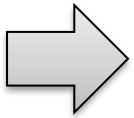
```
push  ebx                       ; save register values
push  ecx
push  esi
mov   esi,OFFSET array          ; starting OFFSET
mov   ecx,LENGTHOF array        ; size, in units
mov   ebx,TYPE array            ; doubleword format
call  DumpMem                   ; display memory
pop   esi                       ; restore register values
pop   ecx
pop   ebx
```

    - Disadvantages
      - It eliminates the performance advantage by avoiding register parameters
      - Programmers must be very careful that each register's PUSH is matched by its appropriate POP

Stack Frames

- 2. Disadvantages of Register Parameters
  - Stack parameters
    - They offer a flexible approach that does not require register parameters

```
push   ebx
push   ecx
push   esi
mov    esi,OFFSET array
mov    ecx,LENGTHOF array
mov    ebx,TYPE array
call   DumpMem
pop    esi
pop    ecx
pop    ebx
```



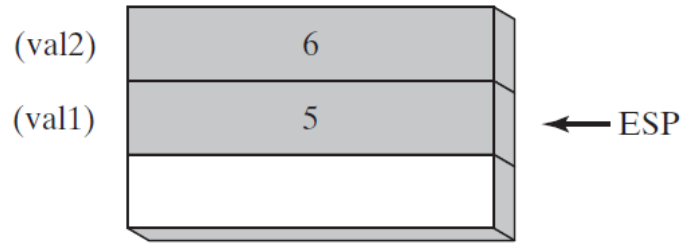
```
push   TYPE array
push   LENGTHOF array
push   OFFSET array
call   DumpMem
```

    - Two general types of arguments are pushed on the stack during subroutine calls:
      - Value arguments (values of variables and constants)
      - Reference arguments (addresses of variables)

Stack Frames

- 2. Disadvantages of Register Parameters
  - Passing by Value
    - A copy of the value is pushed on the stack

```
.data
val1   DWORD 5
val2   DWORD 6
.code
push   val2
push   val1
call   AddTwo
```



- An equivalent function call written in C++ would be

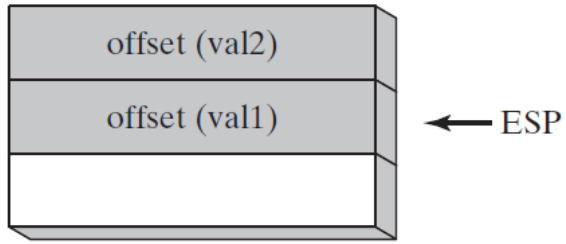
```
int sum = AddTwo( val1, val2 );
```

- The arguments are pushed on the stack in reverse order, which is the standard for the C and C++

Stack Frames

- 2. Disadvantages of Register Parameters
  - Passing by Reference
    - An argument passed by reference consists of the address (offset) of an object

```
.data
val1   DWORD 5
val2   DWORD 6
.code
push   OFFSET val2
push   OFFSET val1
call   Swap
```



    - An equivalent function call written in C++ would be

```
Swap( &val1, &val2 );
```

Stack Frames

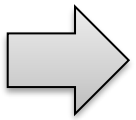
- 2. Disadvantages of Register Parameters
  - Passing Arrays
    - High-level languages always pass arrays to subroutines by reference
      - They push the address of an array on the stack
      - The subroutine can then get the address from the stack and use it to access the array
    - Passing an array by value would require each array element to be pushed on the stack separately
      - Such an operation would be very slow, and it would use up precious stack space

```
.data
array  DWORD 50 DUP(?)
.code
push   OFFSET array
call   ArrayFill
```

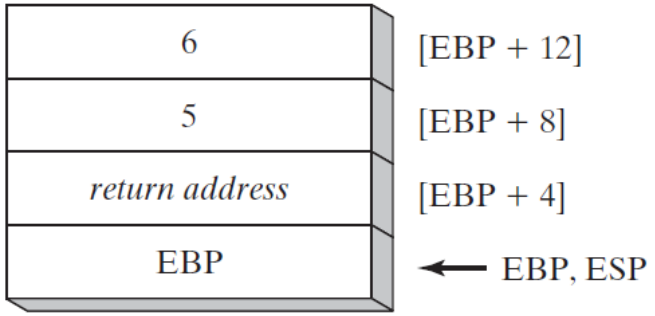
Stack Frames

- 3. Accessing Stack Parameters
  - Initializing and accessing parameters during function calls (C and C++)
    - It begins with 1) saving the EBP register and 2) pointing EBP to the top of the stack
      - Optionally, it may push certain registers on the stack whose values will be restored when the function returns
    - At the end of the function, 1) the EBP register is restored and 2) the RET instruction returns to the caller

```
int AddTwo( int x, int y )
{
     return x + y;
}
```



```
AddTwo PROC
     push  ebp
     mov   ebp,esp
     mov   eax,[ebp + 12]
     add   eax,[ebp + 8]
     pop   ebp
     ret
AddTwo ENDP
```



  - Base-Offset Addressing
    - To access stack parameters, base-offset addressing is used
    - EBP is the base register and the offset is a constant
  - Explicit Stack Parameters
    - Symbolic constants can be used to refer to stack parameters

```
y_param EQU [ebp + 12]
x_param EQU [ebp + 8]

AddTwo PROC
     push  ebp
     mov   ebp,esp
     mov   eax,y_param
     add   eax,x_param
     pop   ebp
     ret
AddTwo ENDP
```

Stack Frames

- 4. 32-Bit Calling Conventions
  - Two most common calling conventions for 32-bit programming in a Windows environment
  1. The C Calling Convention
     - This is used by the C and C++ programming languages
     - Subroutine parameters are pushed on the stack in reverse order
     - When a program calls a subroutine, the combined size of the subroutine parameters is added to the ESP

```
Example1 PROC
     push  6
     push  5
     call  AddTwo
     add   esp,8  ; remove arguments from the stack
     ret
Example1 ENDP
```

     - Programs written in C/C++ always remove arguments from the stack in the calling program after a subroutine has returned

Stack Frames

- 4. 32-Bit Calling Conventions
  - Two most common calling conventions for 32-bit programming in a Windows environment
  2. *STDCALL* Calling Convention
     - An integer parameter is supplied to the RET instruction
       - The integer: the number of bytes of stack space consumed by the procedure's parameter
       - RET adds its operand to ESP

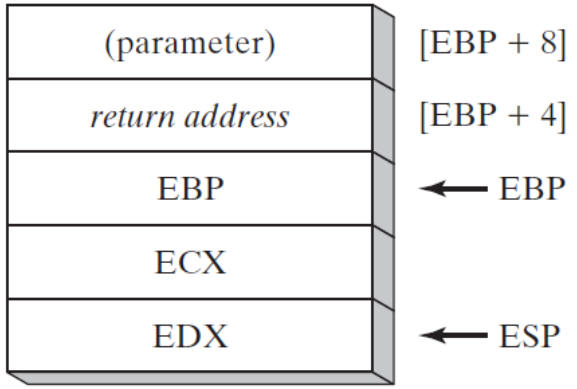
```
AddTwo PROC
    push  ebp
    mov   ebp,esp              ; base of stack frame
    mov   eax,[ebp + 12]       ; second parameter
    add   eax,[ebp + 8]        ; first parameter
    pop   ebp
    ret   8                    ; clean up the stack
AddTwo ENDP
```

     - Like C, STDCALL pushes arguments onto the stack in reverse order
     - By having a parameter in the RET instruction, STDCALL reduces the amount of code generated for subroutine calls and ensures that calling programs will never forget to clean up the stack

Stack Frames

- 4. 32-Bit Calling Conventions
  - Saving and Restoring Registers
    - The registers for being saved should be pushed on the stack just after setting EBP to ESP, and just before reserving space for local variables

```
MySub PROC
     push  ebp                  ; save base pointer
     mov   ebp,esp              ; base of stack frame
     push  ecx
     push  edx                  ; save EDX
     mov   eax,[ebp+8]          ; get the stack parameter
     .
     .
     pop   edx                  ; restore saved registers
     pop   ecx
     pop   ebp                  ; restore base pointer
     ret                        ; clean up the stack
MySub ENDP
```



Stack Frames

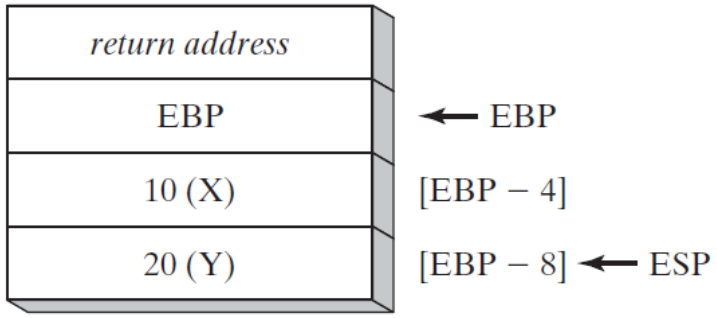
- 5. Local Variables
  - In high-level languages, local variables are created, used, and destroyed within a single subroutine
  - They are created on the runtime stack, usually before the EBP
  - They cannot be assigned default values at assembly time, but they can be initialized at runtime

```
void MySub()
{
    int X = 10;
    int Y = 20;
}
```

Variable	Bytes	Stack Offset
X	4	EBP – 4
Y	4	EBP – 8

  - If this code is compiled into machine language, a total of 8 bytes is reserved for the two variables

```
MySub PROC          (the C calling convention is used)
     push  ebp
     mov   ebp,esp
     sub   esp,8                      ; create locals
     mov   DWORD PTR [ebp-4],10       ; X
     mov   DWORD PTR [ebp-8],20       ; Y
     mov   esp,ebp                    ; remove locals from stack
     pop   ebp
     ret
MySub ENDP
```



  - Before finishing, the function resets the stack pointer by assigning it the value of EBP
    - The effect is to release the local variables from the stack

Stack Frames

- 6. Reference Parameters
  - Reference parameters are usually accessed by procedures using base-offset addressing

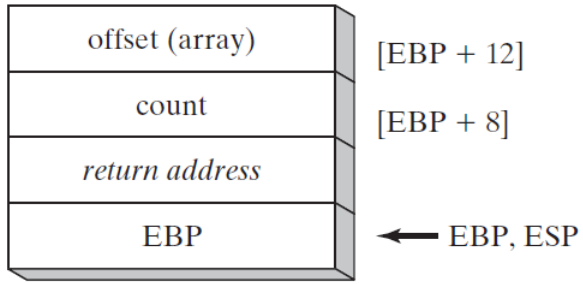
```
mov esi,[ebp+12]  ; points to the array
```

  - ArrayFill Example
    - The ArrayFill procedure fills an array with a pseudorandom sequence of 16-bit integers
    - Two arguments: a pointer to the array (passed by reference) and the array length (passed by value)

```
.data
count = 100
array WORD count DUP(?)

.code
push   OFFSET array
push   count
call   ArrayFill
```

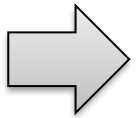
```
ArrayFill PROC
     push  ebp
     mov   ebp,esp
     pushad                      ; save registers
     mov   esi,[ebp+12]          ; offset of array
     mov   ecx,[ebp+8]           ; array length
     cmp   ecx,0                 ; ECX == 0?
     je    L2                    ; yes: skip over loop
L1:
     mov   eax,10000h            ; get random 0 – FFFFh
     call  RandomRange           ; from the link library
     mov   [esi],ax              ; insert value in array
     add   esi,TYPE WORD         ; move to next element
     loop  L1
L2:  popad                       ; restore registers
     pop   ebp
     ret   8                     ; clean up the stack
ArrayFill ENDP
```



Stack Frames

- 7. LEA (Load Effective Address) Instruction
  - The LEA instruction returns the address of an indirect operand
    - The offset is calculated at runtime

```
void makeArray( )
{
     char myString[30];
     for( int i = 0; i < 30; i++ )
          myString[i] = '*';
}
```



```
makeArray PROC
     push  ebp
     mov   ebp,esp
     sub   esp,32                ; myString is at EBP-30
     lea   esi,[ebp-30]          ; load address of myString
     mov   ecx,30                ; loop counter
L1:  mov   BYTE PTR [esi],'*'    ; fill one position
     inc   esi                   ; move to next
     loop  L1                    ; continue until ECX = 0
     add   esp,32                ; remove the array (restore ESP)
     pop   ebp
     ret
makeArray ENDP
```

  - ESP is decremented by 32 to keep it aligned on a doubleword boundary
  - OFFSET operator only works with addresses known at compile time
    - The following statement would not assemble:

```
mov   esi,OFFSET [ebp-30]        ; error
```

Stack Frames

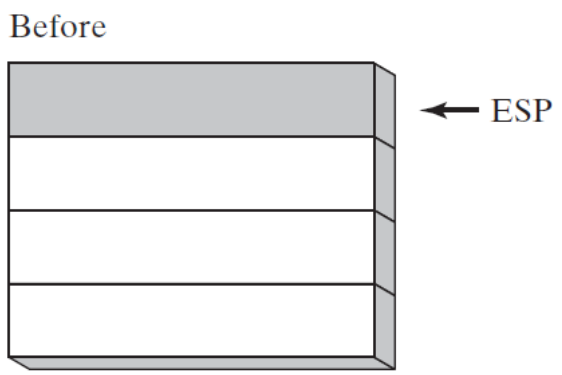
- 8. ENTER and LEAVE Instructions
  - The ENTER instruction automatically creates a stack frame for a called procedure
    - It reserves stack space for local variables and saves EBP on the stack
      - Pushes EBP on the stack (*PUSH EBP*)
      - Sets EBP to the base of the stack frame (*MOV EBP, ESP*)
      - Reserves space for local variables (*SUB ESP, NUMBYTES*)
  - ENTER has two immediate operands

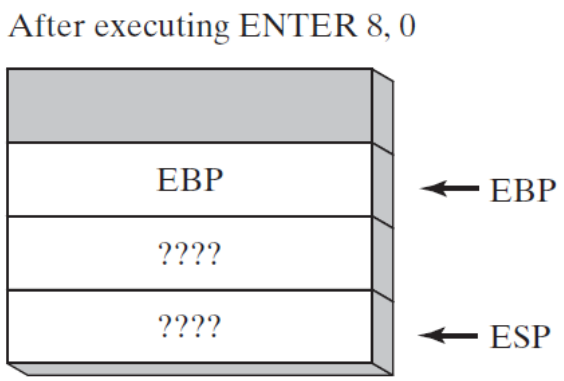
```
ENTER  numbytes, nestinglevel
```

    - The first is a constant specifying the number of bytes of stack space to reserve for local variables
      - Always rounded up to a multiple of 4 to keep ESP on a doubleword boundary
    - The second specifies the lexical nesting level of the procedure

```
MySub PROC          =      MySub PROC
     enter 0,0                 push  ebp
                               mov   ebp,esp
```

```
MySub PROC          =      MySub PROC
     enter 8,0                 push  ebp
                               mov   ebp,esp
                               sub   esp,8
```





Stack Frames

- 8. ENTER and LEAVE Instructions
  - LEAVE Instruction terminates the stack frame for a procedure
    - It reverses the action of a previous ENTER instruction by restoring ESP and EBP to the values they were assigned when the procedure was called

```
MySub PROC
     enter 8,0
     .
     .
     leave
     ret
MySub ENDP
```

=

```
MySub PROC
     push  ebp
     mov   ebp,esp
     sub   esp,8
     .
     .
     mov   esp,ebp
     pop   ebp
     ret
MySub ENDP
```

Recursion

- 1. Recursively Calculating a Sum
  - A recursive procedure named CalcSum, which sums the integers 1 to n
    - n is an input parameter passed in ECX
    - CalcSum returns the sum in EAX
  - The following table shows the return addresses (as labels) pushed on the stack by the CALL instruction, along with the concurrent values of ECX (counter) and EAX (sum)

Pushed on Stack	Value in ECX	Value in EAX
L1	5	0
L2	4	5
L2	3	9
L2	2	12
L2	1	14
L2	0	15

```
INCLUDE Irvine32.inc
.code
main PROC
     mov   ecx,5                ; count = 5
     mov   eax,0                ; holds the sum
     call  CalcSum              ; calculate sum
L1:  call  WriteDec             ; display EAX
     call  Crlf                 ; new line
     exit
main ENDP

;-----------------------------------------------------
CalcSum PROC
; Calculates the sum of a list of integers
; Receives: ECX = count
; Returns: EAX = sum
;-----------------------------------------------------
     cmp   ecx,0                ; check counter value
     jz    L2                   ; quit if zero
     add   eax,ecx              ; otherwise, add to sum
     dec   ecx                  ; decrement counter
     call  CalcSum              ; recursive call
L2:  ret
CalcSum ENDP
end Main
```

Recursion

```
int function factorial(int n)
{
     if(n == 0)
       return 1;
     else
       return n * factorial(n-1);
}
```

- 2. Calculating a Factorial
  - Factorial procedure that uses recursion to calculate a factorial
    - We pass n (an unsigned integer between 0 and 12) on the stack to the Factorial procedure
    - A value is returned in EAX
      - Because EAX is a 32-bit register, the largest factorial it can hold is 12!

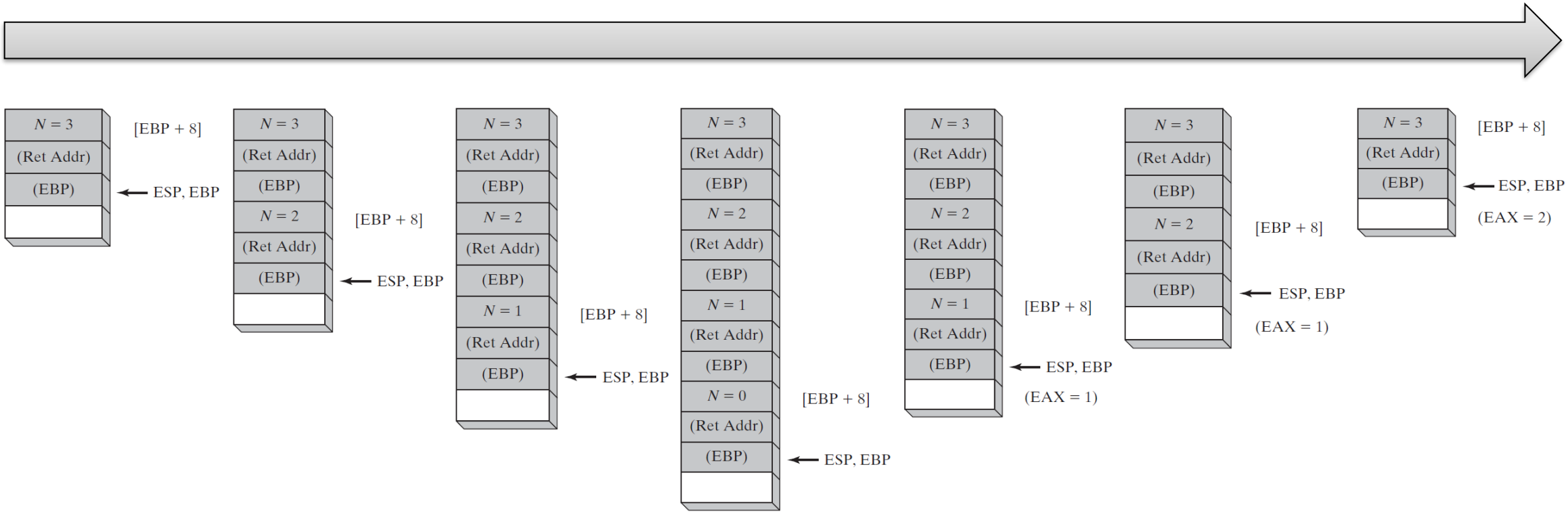
```
INCLUDE Irvine32.inc
.code
main PROC
     push 5                    ; calc 5!
     call Factorial            ; calculate factorial (EAX)
     call WriteDec             ; display it
     call Crlf
     exit
main ENDP
```

```
;-----------------------------------------------------
Factorial PROC
; Receives: [ebp+8] = n, the number to calculate
; Returns: eax = the factorial of n
;-----------------------------------------------------
     push  ebp
     mov   ebp,esp
     mov   eax,[ebp+8]          ; get n
     cmp   eax,0                ; n > 0?
     ja    L1                   ; yes: continue
     mov   eax,1                ; no: return 1 as the value of 0!
     jmp   L2                   ; and return to the caller
L1:  dec   eax
     push  eax                  ; Factorial(n-1)
     call  Factorial
ReturnFact:
     mov   ebx,[ebp+8]          ; get n
     mul   ebx                  ; EDX:EAX = EAX * EBX

L2:  pop   ebp                  ; return EAX
     ret   4                    ; clean up stack
Factorial ENDP
END main
```

Recursion

- 2. Calculating a Factorial
  - Factorial procedure that uses recursion to calculate a factorial
    - We pass n (an unsigned integer between 0 and 12) on the stack to the Factorial procedure
    - A value is returned in EAX
      - Because EAX is a 32-bit register, the largest factorial it can hold is 12!

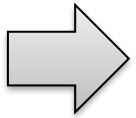


INVOKE, ADDR, PROC, and PROTO

- 1. INVOKE Directive
  - This directive pushes arguments on the stack (in the order specified by the MODEL directive)
  - It then calls a procedure

  - Syntax: `INVOKE procedureName [, argumentList]`

```
push  TYPE array
push  LENGTHOF array
push  OFFSET array
call  DumpArray
```



```
INVOKE DumpArray, OFFSET array, LENGTHOF array, TYPE array
```

(Assumed that STDCALL is in effect)

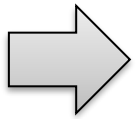
  - EAX, EDX Overwritten
    - If you pass arguments smaller than 32 bits to a procedure, INVOKE causes the assembler to overwrite EAX and EDX when it widens the arguments before pushing them on the stack
    - You can save and restore EAX and EDX before and after the procedure call

INVOKE, ADDR, PROC, and PROTO

- 2. ADDR Operator
  - This operator can be used to pass a pointer argument when calling a procedure using INVOKE
  - The argument passed to ADDR must be an assembly time constant

```
INVOKE FillArray, ADDR myArray
INVOKE mySub, ADDR [ebp+12]     ; error
mov  esi, ADDR myArray          ; error
```

```
.data
Array DWORD 20 DUP(?)
.code
...
INVOKE Swap,
    ADDR  Array,
    ADDR  [Array+4]
```



```
push  OFFSET Array+4
push  OFFSET Array
call  Swap
```

(Assumed that STDCALL is in effect)

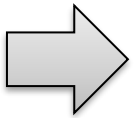
INVOKE, ADDR, PROC, and PROTO

- 3. PROC Directive
  - Syntax of the PROC Directive: `label PROC [attributes] [USES reglist], parameter_list`
  - Parameter Lists
    - The PROC directive permits you to declare a procedure with a comma-separated list of named parameters
    - Your code can refer to the parameters by name rather than by calculated stack offsets such as [ebp+8]

```
label PROC [attributes] [USES reglist],
    parameter_1,
    parameter_2,
    .
    .
    parameter_n
```

  - A single parameter has the following syntax: `paramName:type`

```
AddTwo PROC,
     val1:DWORD,
     val2:DWORD
     mov   eax,val1
     add   eax,val2
     ret
AddTwo ENDP
```



```
AddTwo PROC
     push  ebp
     mov   ebp, esp
     mov   eax,dword ptr [ebp+8]
     add   eax,dword ptr [ebp+0Ch]
     leave
     ret   8
AddTwo ENDP
```

INVOKE, ADDR, PROC, and PROTO

- 3. PROC Directive
  - RET Instruction Modified by PROC
    - When PROC is used with one or more parameters and STDCALL is the default protocol, MASM generates the following entry and exit code, assuming PROC has n parameters:

```
push  ebp
mov   ebp,esp
.
.
leave
ret   (n*4)
```

  - Specifying the Parameter Passing Protocol
    - The attributes field of the PROC directive lets you specify the language convention for passing parameters
    - It overrides the default language convention specified in the .MODEL directive

```
Example1 PROC C,
    parm1:DWORD, parm2:DWORD
```

```
Example1 PROC STDCALL,
    parm1:DWORD, parm2:DWORD
```

INVOKE, ADDR, PROC, and PROTO

- 4. PROTO Directive
  - The PROTO directive creates a prototype for an existing procedure
  - A prototype declares a procedure's name and parameter list
    - It allows you to call a procedure before defining it
  - MASM requires a prototype for each procedure called by INVOKE
    - PROTO must appear first before INVOKE

```
MySub PROTO     ; procedure prototype
      .
INVOKE MySub    ; procedure call
      .
MySub PROC      ; procedure implementation
      .
      .
MySub ENDP
```

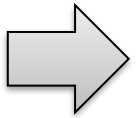
INVOKE, ADDR, PROC, and PROTO

- 4. PROTO Directive
  - The PROTO directive creates a prototype for an existing procedure
  - A prototype declares a procedure's name and parameter list
    - It allows you to call a procedure before defining it

```
Sub1 PROTO, p1:BYTE, p2:WORD, p3:PTR BYTE

.data
byte_1   BYTE  10h
word_1   WORD  2000h
word_2   WORD  3000h
dword_1  DWORD 12345678h

INVOKE Sub1, byte_1, word_1, ADDR byte_1
```



```
push  404000h                    ; ptr to byte_1
sub   esp,2                      ; pad stack with 2 bytes
push  word ptr ds:[00404001h]    ; value of word_1
mov   al,byte ptr ds:[00404000h] ; value of byte_1
push  eax
call  00401071
```